Software Requirements Specification

Writer's Creative Ideas Information System

Academic / Engineering Project

Revised requirements baseline
Version 2.0

Document status	Revised requirements baseline
Methodology	Socratic Knowledge Elicitation & Documentation (SKED)
Guideline	IEEE 830-style SRS structure
Target product	Client-server application with local-first mobile client

Contents

1 Introduction **3**

1.1 Purpose . . . 3

1.2 Scope . . . 3

1.3 Intended Audience and Reading Guidance . . . 3

1.4 Definitions . . . 3

1.5 References . . . 4

1.6 Document Conventions . . . 4

2 Overall Description **4**

2.1 Product Perspective . . . 4

2.2 Product Functions . . . 5

2.3 User Classes and Characteristics . . . 5

2.4 Operating Environment . . . 5

2.5 Design and Implementation Constraints . . . 5

2.6 User Documentation . . . 6

2.7 Assumptions and Dependencies . . . 6

3 Specific Requirements **6**

3.1 External Interface Requirements . . . 6

3.1.1 User Interface . . . 6

3.1.2 Software Interfaces . . . 6

3.1.3 Communications Interfaces . . . 6

3.1.4 Data and Export Interface . . . 6

3.2 Functional Requirements . . . 7

3.2.1 Quick Capture and Entry Management . . . 7

3.2.2 Books, Chapters, and Associations . . . 7

3.2.3 Entry Linking . . . 8

3.2.4 Retrieval and Search . . . 8

3.2.5 Export . . . 8

3.2.6 API and Synchronization . . . 9

3.3 Non-Functional Requirements . . . 10

4 Acceptance Criteria **11**

4.1 Functional Acceptance Criteria . . . 12

4.2 API and Synchronization Acceptance Criteria . . . 13

4.3 Non-Functional Acceptance Criteria . . . 14

5 Behavioral Scenarios (Given–When–Then) **15**

6 Traceability and Outstanding Specification Details **17**

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the agreed scope, externally observable behavior, quality attributes, acceptance criteria, and behavioral scenarios for the Writer's Creative Ideas Information System. It is intended for the project owner, analysts, designers, implementers, testers, and academic reviewers. Requirement identifiers are stable references for specifications, tickets, tests, and implementation traceability.

1.2 Scope

The system is a client-server application for capturing, organizing, linking, searching, and exporting literary creative material. A mobile or web client communicates with a backend server through a versioned REST API. The client stores a local SQLite working copy and an outgoing change queue so that capture, organization, search of locally available data, and export of locally available data continue while offline. Changes are synchronized with the server when connectivity is available.

Users manage Books and Chapters, create Entries with preset Entry Types and Tags, associate Entries with Books and Chapters, link Entries to one another, and synchronize their data. Server data is authoritative for successfully synchronized changes; unsynchronized local changes remain available on the originating client until accepted or explicitly resolved.

The MVP includes a mobile client for iOS and Android. A web client is an allowed client of the same API, but its supported browser matrix and release commitment are not specified in this baseline. The system does not provide a manuscript editor, multi-user collaboration on the same project, publishing workflow, or complex project/deadline management. Text export to device-local storage is included. User registration, identity-provider choice, and account recovery are not specified; API access is restricted to an authenticated user context.

1.3 Intended Audience and Reading Guidance

Section 2 describes context, users, platform, and constraints. Section 3 contains the normative functional, API, synchronization, and non-functional requirements. Section 4 defines acceptance evidence for every requirement. Section 5 specifies representative Given–When–Then (GWT) behavioral scenarios. Requirements use the word "shall" to indicate a mandatory product obligation.

1.4 Definitions

Book
: A user-created container for creative material belonging to a literary project.

Chapter
: A named subdivision of a Book. Entries may be associated with a Chapter and with the Chapter's parent Book.

Entry
: A piece of creative material managed by the system. "Note" is a user-facing synonym, not a separate data entity.

Entry Type
: The preset primary classification of an Entry: Character, Scene, Plot Idea, World-building, Dialogue/Phrase, or General Note.

Tag A user-applied flexible label that supplements, and does not replace, an Entry Type.

Standalone Entry
: An Entry with no current Book or Chapter associations.

Direct Book Association
: A relationship explicitly created between an Entry and a Book as a whole.

Chapter Association
: A relationship between an Entry and a Chapter. It automatically implies association with the Chapter's parent Book. A direct Book Association is tracked separately.

Entry Link
: An unlabeled, symmetric relationship between two Entries, distinct from Book and Chapter associations.

Quick Capture
: The workflow for creating an Entry with minimal required input. A General Note can be created with Content only.

Display Title
: The title shown for an Entry. It is the explicit Title when nonblank; otherwise, it is derived from Content under REQ-F-003.

Book Project Export
: A text export of a Book and the Entries associated with it directly or through its Chapters.

Local change queue / outbox
: Durable client-side records of local changes not yet acknowledged by the server.

Sync cursor
: An opaque server-issued token that lets a client request changes made after its last successful download.

Tombstone
: A synchronization record indicating deletion of an entity or relationship, retained long enough to propagate that deletion to clients.

Revision
: A server-issued version identifier for an entity, used to detect updates based on stale data.

1.5 References

- IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*; used as a structural guideline. This document is not a claim of formal IEEE compliance or certification.
- Project concept and prior approved domain requirements, revised here for the client-server architecture.

1.6 Document Conventions

Identifiers beginning `REQ-F-` denote domain functional requirements, `REQ-API-` denote API and synchronization requirements, and `REQ-NF-` denote non-functional requirements. Acceptance criteria use corresponding `AC-F-`, `AC-API-`, and `AC-NF-` identifiers. GWT scenarios complement the individual criteria and do not replace them. Section 3.3 defines the minimum API surface for the upcoming OpenAPI and Mock API work; detailed wire schemas are to be finalized in that contract.

2 Overall Description

2.1 Product Perspective

The product consists of a client and a backend. The mobile and/or web client provides the user interface, local SQLite cache, durable change outbox, offline workflows, and local text export. The backend exposes a versioned REST API, validates requests, authorizes access to user-owned data, persists synchronized records, and provides incremental change feeds. The backend is authoritative for acknowledged changes; the local database remains the client's working store. A live connection is required to synchronize changes and retrieve uncached server data, but not to complete core workflows

over data already stored on the client.

2.2 Product Functions

At a high level, the product provides:

- quick capture and lifecycle management of Entries;
- preset Entry Types and user-defined Tags;
- Book and Chapter organization, including many-to-many Entry-to-Book association;
- automatic parent-Book association for Chapter-associated Entries;
- symmetric links between Entries;
- case-insensitive full-text search and combined search/type/tag filtering;
- local text export of an Entry or Book project; and
- durable offline edits and synchronization with the backend over REST; and
- internal diagnostic logging without a dedicated user-facing log interface.

2.3 User Classes and Characteristics

The primary user class is an individual writer, novelist, screenwriter, or storyteller who needs to capture ideas on the move and later organize and retrieve a large volume of heterogeneous creative material. Each server-side record belongs to an authenticated user context. Collaboration and shared editing between multiple users are not included in this baseline.

2.4 Operating Environment

- Mobile application developed with React Native, Expo, and TypeScript.
- Android 10.0 or later (API level 29 or later).
- iOS 15.0 or later.
- Minimum supported logical viewport: 360 by 640 device-independent pixels (dp).
- On-device SQLite persistence using `expo-sqlite`.
- Backend reachable over HTTPS for synchronization and uncached remote data.
- JSON request and response bodies for the REST API.
- Core client workflows for locally available data must function without internet connectivity.

2.5 Design and Implementation Constraints

- The mobile client is in scope; the same API may serve a web client. Browser support and web UI requirements remain to be decided before a web release.
- The client shall persist working data and pending changes locally in SQLite. Server data is authoritative for successfully synchronized changes.
- The API shall use HTTPS, JSON, RESTful resource semantics, and a versioned base path.
- API authorization shall scope all reads and writes to the authenticated user's data. The authentication mechanism and account lifecycle are design decisions for the API/security design stage.
- The Entry Types are a fixed preset set. Flexible categorization is provided by Tags, not user-created Entry Types.
- Book and Chapter deletion must not cascade into deletion of Entries.
- Entry Links are unlabeled and symmetric.
- The codebase shall use TDD and meet the coverage thresholds in REQ-NF-014.
- Detailed API schemas, authentication flow, deployment topology, synchronization tombstone retention, and conflict-resolution UI are design-stage decisions and shall not weaken the requirements in this SRS.

2.6 User Documentation

In-app help content, onboarding, and end-user documentation are not specified by the approved MVP decisions. Their inclusion and format remain for project planning.

2.7 Assumptions and Dependencies

- The device has functional local storage and an operating-system-supported means of saving or selecting a local destination for exported files.
- Availability of a file destination or permission may vary by platform; export failure must be reported as specified.
- Network access and backend availability can be intermittent; offline changes must survive application closure and restart.
- API service availability, latency targets, hosting topology, and supported web browsers are not specified by this baseline.
- Exact export layout, diagnostic-log redaction and timestamp precision, and the precise code scope included in the coverage threshold require verification/design decisions.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interface

The interface shall be designed for small smartphone screens and quick on-the-go capture. It shall provide workflows to create and manage Entries, Books, Chapters, Tags, and Entry Links; retrieve Entries; review synchronization status and errors; and initiate exports. Detailed screen layouts, navigation structure, and visual design are outside this SRS. The interface shall not imply that a pending local change has been accepted by the server.

3.1.2 Software Interfaces

The mobile client shall use React Native, Expo, TypeScript, and `expo-sqlite` for local relational persistence. The client shall communicate with the backend using the REST API specified in Section 3.3. It shall use platform capabilities as needed for local text-file export and accessibility. The backend's technology stack is not prescribed.

3.1.3 Communications Interfaces

Client-server requests shall use HTTPS. The API shall exchange JSON representations using UTF-8 and shall follow RESTful resource-oriented principles. The API base path shall be `/api/v1`. Requests and responses shall use appropriate HTTP methods, status codes, and content types. No plaintext HTTP API is permitted for production communication. Offline client operations shall not present a network request as successful when the server has not acknowledged it.

3.1.4 Data and Export Interface

The system data model shall represent Entries, Books, Chapters, Tags, Book Associations, Chapter Associations, Entry Links, synchronization metadata, and diagnostic events. The client shall persist the local working copy and outbox in SQLite. Export output shall be a `.txt` file saved to device-local storage. UTF-8 encoding is required. Exact export metadata, punctuation, and file-naming conventions remain unspecified.

3.2 Functional Requirements

3.2.1 Quick Capture and Entry Management

ID	Requirement
REQ-F-001	The system shall allow the user to create an Entry through a Quick Capture workflow.
REQ-F-002	The system shall require nonblank Content to create an Entry. Title shall be optional.
REQ-F-003	When Title is blank, the system shall derive the Display Title from Content. The 25-character limit applies only to the body content of the Display Title; an ellipsis ("...") is additional. If truncation occurs because Content exceeds 25 characters or because a newline occurs within the first 25 characters, the system shall append "..." after the retained body content. Content after the first newline shall not appear in the Display Title.
REQ-F-004	The system shall provide exactly these preset Entry Types: Character, Scene, Plot Idea, World-building, Dialogue/Phrase, and General Note.
REQ-F-005	The system shall allow the user to create a General Note with Content as the only required input.
REQ-F-006	The system shall allow the user to view an Entry's Content, explicit Title or generated Display Title, Entry Type, Tags, Book Associations, Chapter Associations, and Entry Links.
REQ-F-007	The system shall allow the user to edit an Entry's Content and Title, change its Entry Type, and add or remove its Tags, Book Associations, and Chapter Associations.
REQ-F-008	The system shall allow the user to delete an Entry. Deleting an Entry shall not leave active Entry Links to it.
REQ-F-009	The system shall allow the user to apply Tags to Entries and remove Tags from Entries.
REQ-F-010	The system shall allow an Entry to have no Book or Chapter Associations and remain available as a Standalone Entry for later association.

3.2.2 Books, Chapters, and Associations

ID	Requirement
REQ-F-011	The system shall allow the user to create, view, edit, and delete Books.
REQ-F-012	The system shall allow the user to create, view, edit, and delete Chapters within a Book.
REQ-F-013	The system shall allow the user to directly associate an Entry with a Book as a whole and remove that Book Association.
REQ-F-014	The system shall allow an Entry to have Book Associations with multiple Books.
REQ-F-015	The system shall allow the user to associate an Entry with a Chapter within a Book and remove that Chapter Association.
REQ-F-016	Associating an Entry with a Chapter shall automatically associate the Entry with that Chapter's parent Book. The system shall distinguish this Chapter-derived association from a separately created direct Book Association.

ID	Requirement
REQ-F-017	Deleting a Book or Chapter shall not delete associated Entries. The system shall remove associations to the deleted Book or Chapter while preserving associations to other extant Books or Chapters. An Entry with no remaining associations shall remain available as a Standalone Entry.
REQ-F-018	Removing or deleting a Chapter Association shall preserve a separately created direct Book Association to the Chapter's parent Book. A Book association that exists only because of that Chapter Association shall not remain solely by virtue of the deleted Chapter Association.

3.2.3 Entry Linking

ID	Requirement
REQ-F-019	The system shall allow the user to link two Entries without assigning a label to the Entry Link.
REQ-F-020	Entry Links shall be symmetric: linking Entry A to Entry B shall make the link visible from both Entries.
REQ-F-021	The system shall allow the user to remove an Entry Link. Removing the link shall not delete either Entry.

3.2.4 Retrieval and Search

ID	Requirement
REQ-F-022	The system shall provide case-insensitive full-text search across both Entry Title and Entry Content and display matching Entries. A generated Display Title shall be searchable as the Entry's displayed title.
REQ-F-023	The system shall allow the user to filter Entries by Entry Type. When multiple Entry Types are selected, an Entry shall match all selected Entry Types (strict AND logic).
REQ-F-024	The system shall allow the user to filter Entries by Tag. When multiple Tags are selected, an Entry shall match all selected Tags (strict AND logic).
REQ-F-025	When Search, Entry Type, and Tag criteria are combined, the system shall use strict AND logic across categories: a result shall satisfy every active criterion. Strict AND logic shall also apply among multiple selected Entry Types and among multiple selected Tags.

3.2.5 Export

ID	Requirement
REQ-F-026	The system shall allow the user to export an Entry as a `.txt` file to device-local storage. The exported file shall contain the Entry's Content.
REQ-F-027	The system shall allow the user to export a Book project as a `.txt` file to device-local storage.

ID	Requirement
REQ-F-028	A Book Project Export shall include Entries associated directly with the Book and Entries associated with any of its Chapters. Chapter-associated Entries shall appear under their respective Chapter headings. Chapters shall be ordered by an internal integer position field in ascending order when that field is available; otherwise, they shall be ordered by `created_at` ascending. Direct Book-level Entries not placed under a Chapter shall appear in a dedicated "General Book Notes" section at the end.
REQ-F-029	Each Entry shall appear no more than once in a Book Project Export. An Entry associated with a Chapter and its parent Book shall appear under the Chapter heading and not be duplicated in "General Book Notes".

3.2.6 API and Synchronization

The routes below define the required logical API surface. Exact JSON schemas, field optionality, and generated-client details shall be finalized in the OpenAPI deliverable required by REQ-API-014.

ID	Requirement
REQ-API-001	The backend shall expose a versioned REST API under `/api/v1`, communicate using HTTPS, accept and return UTF-8 JSON representations, and use resource-oriented URIs and appropriate HTTP methods and status codes.
REQ-API-002	The backend shall require a server-recognized authenticated user context for protected API operations and shall authorize each read and write against that user's data. A user shall not be able to read or modify another user's records by changing an identifier or request parameter. The authentication mechanism is not prescribed by this SRS.
REQ-API-003	The API shall provide create, read, update, and delete operations for Entries through `/entries` and `/entries/{entryId}`. Entry representations shall support the data needed by REQ-F-002 through REQ-F-009, including stable identifiers and server revision metadata.
REQ-API-004	The API shall provide create, read, update, and delete operations for Books through `/books` and `/books/{bookId}`. Deleting a Book shall follow REQ-F-017 and shall not delete Entries.
REQ-API-005	The API shall provide create, read, update, and delete operations for Chapters through `/books/{bookId}/chapters` and `/books/{bookId}/chapters/{chapterId}`. A Chapter shall belong to one Book; deleting a Chapter shall follow REQ-F-017 and REQ-F-018.
REQ-API-006	The API shall provide Tag creation and retrieval through `/tags` and `/tags/{tagId}`, and shall allow Tag assignment and removal for an Entry through `/entries/{entryId}/tags` and `/entries/{entryId}/tags/{tagId}`.
REQ-API-007	The API shall allow clients to create and remove direct Book Associations and Chapter Associations through `/entries/{entryId}/books/{bookId}` and `/entries/{entryId}/chapters/{chapterId}`. It shall preserve the distinction between direct and Chapter-derived Book Associations and enforce the rules in REQ-F-013 through REQ-F-018.
REQ-API-008	The API shall allow clients to create and remove Entry Links through `/entries/{entryId}/links/{linkedEntryId}`. A link shall be symmetric, unlabeled, and removed from both Entries when deleted; deleting an Entry shall not leave active links to it.

ID	Requirement
REQ-API-009	The API shall support list retrieval and the search and filtering behavior in REQ-F-022 through REQ-F-025 for Entries. Collection responses shall support bounded pagination and return enough continuation information to retrieve subsequent pages.
REQ-API-010	The API shall provide synchronization operations at `POST /sync/changes` and `GET /sync/changes`. Upload shall accept client-generated stable entity identifiers and an ordered batch of pending create, update, delete, and relationship changes. Download shall accept an opaque cursor and return changes since that cursor plus a new cursor. Deletions shall be represented by tombstones so that they propagate to clients that were offline.
REQ-API-011	The client shall persist local domain changes and pending sync operations in SQLite before reporting a local save as complete. It shall support creating, editing, associating, searching locally available data, and exporting locally available data while offline; retain the outbox across app restart; and attempt synchronization when connectivity returns.
REQ-API-012	Sync operations shall be idempotent by operation identifier so that retrying an operation after a lost response does not duplicate or apply it twice. Each update shall carry its base server revision; the server shall detect stale revisions and return a conflict with the current server revision/representation rather than silently overwriting either version. The client shall retain the unsynchronized local change and expose the conflict state; conflict resolution policy and UI are deferred to design.
REQ-API-013	The API shall validate request data and return machine-readable JSON errors with an appropriate HTTP status for invalid input, missing resources, unauthenticated/unauthorized access, revision conflicts, and server failures. A failed operation shall not be reported as successful. A synchronization response shall identify the outcome of each submitted operation so the client can retain rejected or conflicted changes without resubmitting acknowledged operations as new changes.
REQ-API-014	Before API implementation and Mock API acceptance, the project shall provide an OpenAPI specification covering every required route, method, request and response schema, authentication/security declaration, status/error response, pagination behavior, revision/conflict semantics, and sync cursor/tombstone behavior in REQ-API-001 through REQ-API-013. The specification shall be sufficient to generate or configure representative mock responses for the required workflows.

3.3 Non-Functional Requirements

ID	Requirement
REQ-NF-001	On a supported iOS 15+ simulator or Android 10+ emulator with a local SQLite database pre-populated with 1,000 Entries, the client shall save a new Entry locally and confirm that local save within 1 second, without waiting for a server response.
REQ-NF-002	On the same supported environments and dataset, the client shall return local search and filter results within 2 seconds.

ID	Requirement
REQ-NF-003	Without internet connectivity, core capture, organization, search, and export workflows shall operate on data already available in the client's local store. Remote synchronization and retrieval of uncached server data may be unavailable until connectivity returns.
REQ-NF-004	The client shall persist its working data, sync metadata, and pending outbox in on-device SQLite using `expo-sqlite`. Locally saved data and pending changes shall remain available after the application is closed and reopened. Successfully synchronized data shall be retained in the local cache unless removed by the user or a defined cache policy.
REQ-NF-005	The system shall preserve valid relationships among Entries, Books, Chapters, Tags, Book Associations, Chapter Associations, and Entry Links when records or associations are created, edited, detached, or deleted.
REQ-NF-006	Deleting a Book or Chapter shall not cause loss of the Content or identity of any associated Entry.
REQ-NF-007	The application shall install and operate on Android 10.0 or later (API 29 or later) and iOS 15.0 or later.
REQ-NF-008	Core workflows shall be usable at a minimum viewport of 360 by 640 dp without clipped interactive controls or unintended horizontal scrolling.
REQ-NF-009	All interactive UI controls shall have explicit React Native `accessibilityLabel` attributes. The interface shall support platform text scaling without making core workflows inaccessible.
REQ-NF-010	Exported `.txt` files shall use UTF-8 encoding and preserve representative accented and non-Latin text.
REQ-NF-011	If an export cannot be completed, the system shall notify the user, shall not report success, and shall not present a partial file as a completed export.
REQ-NF-012	The client shall maintain an internal, device-local diagnostic/audit log recording the occurrence and time of Entry, Book, and Chapter creation, update, and deletion; association and Entry Link changes; export operations; and sync attempts/outcomes. Timestamps shall use ISO-8601 format in UTC. The log shall have a maximum size of 2 MB and shall rotate on a first-in, first-out (FIFO) basis by removing the oldest log records when necessary to remain within the limit.
REQ-NF-013	The MVP shall not provide a dedicated user-facing interface for viewing or managing the diagnostic log.
REQ-NF-014	The codebase shall be developed using Test-Driven Development (TDD). Domain logic, client SQLite repositories, sync logic, and backend API behavior shall have automated unit and integration tests with at least 85 percent statement and branch coverage, measured using Jest with `npm test --coverage` (excluding `*.config.js` and `*.d.ts`). Additionally, the engineering process shall maintain project logs recording requirement conflict resolution, coding-agent (AI co-pilot) interaction prompts and decisions, and key technical trade-offs alongside technical documentation in the project repository.

4 Acceptance Criteria

Each criterion below states observable evidence for its associated requirement. Criteria are concise verification conditions and are not written in GWT form.

4.1 Functional Acceptance Criteria

ID	Acceptance criterion
AC-F-001	A Quick Capture action creates an Entry, and the Entry is available in the Entry list.
AC-F-002	Nonblank Content with no Title can be saved. Blank Content is rejected with a validation message.
AC-F-003	With blank Title, Content of 25 characters or fewer and no newline displays unchanged. When Content exceeds 25 characters, the first 25 body characters are followed by "...". When a newline occurs within the first 25 characters, the body ends before the newline and "..." is appended; no text from subsequent lines appears. The ellipsis does not count toward the 25-character body limit.
AC-F-004	The type selector offers the six specified preset types, and each can be assigned to an Entry.
AC-F-005	A General Note can be saved with Content as the only supplied field.
AC-F-006	Viewing an Entry displays its Content, explicit or generated title, Entry Type, Tags, Book and Chapter Associations, and Entry Links.
AC-F-007	Changes to an Entry's Content, Title, Entry Type, Tags, and Book or Chapter Associations persist and appear on subsequent viewing.
AC-F-008	A deleted Entry is absent from lists and search results, and no active Entry Link points to it.
AC-F-009	A Tag can be added to an Entry, appears in its details, and can be removed.
AC-F-010	An unassociated Entry remains available as standalone and can subsequently be associated with a Book or Chapter.
AC-F-011	The user can create, view, edit, and delete Books. Book deletion leaves associated Entries intact.
AC-F-012	The user can create, view, edit, and delete Chapters within a Book. Chapter deletion leaves associated Entries intact.
AC-F-013	A direct Book Association can be created, displayed, and removed.
AC-F-014	One Entry can be associated with at least two Books and appears under each.
AC-F-015	A Chapter Association can be created, displayed, and removed.
AC-F-016	Creating a Chapter Association also creates the parent Book association; the data distinguishes this derived relation from a direct Book Association.
AC-F-017	Deleting a Book or Chapter leaves Entry content intact, removes associations to deleted records, preserves remaining valid associations, and leaves Entries with no associations available as standalone.
AC-F-018	Removing a Chapter Association preserves an independent direct Book Association. A Book association that existed only through the Chapter is removed when that Chapter association is removed and no other relation implies it.
AC-F-019	Two Entries can be linked without a label; linking does not alter their Entry content or type.
AC-F-020	After linking Entries A and B, each Entry displays the other as linked.
AC-F-021	Removing the link from either Entry removes it from both link lists; both Entries remain available.
AC-F-022	A case-insensitive query matching text in either Title or Content returns the Entry; a query with no match returns no such Entry.

ID	Acceptance criterion
AC-F-023	Applying an Entry Type filter returns matching Entries and excludes Entries of other types.
AC-F-024	Applying a Tag filter returns Entries with the selected Tag. When multiple Tags are selected, only Entries with every selected Tag are returned.
AC-F-025	With Search, Type, and Tag active, every result matches all active criteria; an Entry failing any criterion is excluded. When multiple Entry Types or Tags are selected, a result matches every selected value within that category.
AC-F-026	Export creates a local `.txt` file containing the Entry's Content.
AC-F-027	Export creates a local `.txt` file for the selected Book project.
AC-F-028	The export includes directly associated and Chapter-associated Entries; Chapter entries occur beneath their Chapter headings ordered by ascending internal integer position when available, otherwise by ascending `created_at`. Direct Book-level Entries not placed under Chapters occur under "General Book Notes" at the end.
AC-F-029	An Entry associated with both a Chapter and its parent Book occurs once, under its Chapter heading, and not again in "General Book Notes".

4.2 API and Synchronization Acceptance Criteria

ID	Acceptance criterion
AC-API-001	API contract and integration checks show the `/api/v1` base path, HTTPS deployment configuration, UTF-8 JSON media types, resource-oriented routes, and appropriate method/status behavior.
AC-API-002	Requests without an authenticated context are rejected. A user cannot retrieve or modify another user's records by substituting record IDs or query parameters.
AC-API-003	Contract/integration tests create, retrieve, update, and delete an Entry through the specified collection and item routes; the representation carries a stable identifier and revision.
AC-API-004	Contract/integration tests create, retrieve, update, and delete a Book; deletion leaves associated Entries intact.
AC-API-005	Contract/integration tests manage Chapters under their parent Book; a Chapter cannot be managed under a different Book, and its deletion follows the safe-detach rules.
AC-API-006	Contract/integration tests create and retrieve Tags, assign a Tag to an Entry, retrieve the assignment, and remove it.
AC-API-007	Contract/integration tests create and remove direct Book and Chapter Associations and verify that derived and direct Book Associations remain distinguishable and obey REQ-F-016 through REQ-F-018.
AC-API-008	Contract/integration tests link two Entries, observe the link from both Entries, remove it from either endpoint, and verify it is absent from both.
AC-API-009	API tests verify case-insensitive search, strict AND behavior for combined filters, and retrieval of all matching records through bounded pages.
AC-API-010	A sync test uploads create, update, relationship, and delete changes using stable client IDs, then downloads the resulting changes with a cursor. A deletion appears as a tombstone, and a subsequent cursor returns only changes after the previous cursor.

ID	Acceptance criterion
AC-API-011	With networking disabled, a local mutation is acknowledged after durable local persistence, survives app restart in the outbox, and is uploaded after connectivity returns; export and search of local data remain available offline.
AC-API-012	Replaying an operation ID does not duplicate its effect. A stale base revision receives a conflict response with current server data, while the client retains its unsynchronized local value and exposes a conflict state.
AC-API-013	Invalid, missing, unauthorized, and conflicting requests receive documented machine-readable JSON errors and appropriate HTTP statuses. Sync responses distinguish accepted, rejected, and conflicted operations, and retrying a failed response does not mark it as accepted.
AC-API-014	The OpenAPI document validates and contains every required operation, schema, security declaration, status/error response, pagination rule, revision/conflict rule, cursor rule, and tombstone rule. A mock can return representative success, validation, authorization, and conflict responses for each applicable workflow.

4.3 Non-Functional Acceptance Criteria

ID	Acceptance criterion
AC-NF-001	On an iOS 15+ simulator or Android 10+ emulator with a local SQLite database pre-populated with 1,000 Entries, local Entry save and confirmation complete within 1 second without waiting for the API.
AC-NF-002	On the same environments and dataset, local search and filter results appear within 2 seconds.
AC-NF-003	With network connectivity disabled, capture, organization, search, and export complete using locally stored data; attempts to retrieve uncached server data may report unavailable.
AC-NF-004	Inspection confirms use of `expo-sqlite` for the local working copy and outbox; saved data and pending changes remain after closing and reopening the app.
AC-NF-005	Tests of create, edit, associate, detach, and delete operations show no dangling references and show only valid remaining relationships.
AC-NF-006	After Book or Chapter deletion, associated Entry identity and Content remain retrievable.
AC-NF-007	The release application installs and launches on Android API 29+ and iOS 15+.
AC-NF-008	At 360 by 640 dp, core workflows have no clipped interactive controls or unintended horizontal scrolling.
AC-NF-009	Every interactive UI control has an explicit React Native `accessibilityLabel` attribute; platform text scaling does not block completion of core workflows.
AC-NF-010	Exported files decode as UTF-8 and preserve representative accented and non-Latin test strings.
AC-NF-011	A simulated or induced export failure produces an error indication, no success indication, and no partial file reported as complete.
AC-NF-012	Inspection confirms that each required event, including sync attempt/outcome, is timestamped in ISO-8601 UTC format and that log rotation retains no more than 2 MB of records by removing the oldest records first.
AC-NF-013	The MVP contains no dedicated user-facing screen or controls for viewing or managing diagnostic logs.

ID	Acceptance criterion
AC-NF-014	Development evidence demonstrates a tests-first workflow with at least 85 percent statement and branch coverage reported by Jest for domain logic, client SQLite repositories, sync logic, and backend API behavior. Repository audit also confirms the presence of engineering logs documenting requirement conflict resolution, coding-agent prompts and responses, and technical trade-offs.

5 Behavioral Scenarios (Given–When–Then)

The following scenarios are representative acceptance-level examples. They complement, but do not exhaust, the acceptance criteria above.

Scenario 1: Generate a Display Title

Given an Entry is created with a blank Title
When the user saves Content
Then the system generates a Display Title whose body is limited to the first 25 characters
And if the Content exceeds 25 characters or a newline occurs within the first 25 characters, the system appends "..." after the retained body, outside the 25-character limit
And no text after the first newline appears in the Display Title.

Scenario 2: Create an Entry with All Details

Given the user is creating an Entry and has entered an explicit Title and Content, selected an Entry Type, and added Tags
When the user saves the Entry
Then the Entry is saved with the entered Title and Content, selected Entry Type, and Tags
And viewing the Entry shows those saved values without replacing the explicit Title with a generated one.

Scenario 3: Chapter and Book Associations

Given an Entry's only Chapter Association in Book A is to the Chapter being deleted, and it is also directly associated with Book B
When the user deletes that Chapter
Then the Entry remains available and its association with Book B remains
And it is no longer associated with the deleted Chapter or its Chapter-derived Book A association.

Given an Entry has no association other than one with a Chapter
When the user deletes that Chapter
Then the Entry remains available as a Standalone Entry.

Given an Entry is associated with a Book
When the user deletes that Book
Then the Entry remains available and is detached from the deleted Book.

Scenario 4: Symmetric Entry Links

Given Entries A and B exist and are not linked
When the user links A to B
Then B appears in A's links and A appears in B's links.

When the user removes the link from either Entry
Then the link no longer appears for either Entry, and both Entries remain available.

Scenario 5: Search and Combined Filters

Given Entries contain matching text in their Titles or Contents using different letter cases
When the user searches for that text
Then all Entries with a case-insensitive match in either field appear, and Entries without a match do not.

Given Entries have different Entry Types and Tags
When the user applies a search term, multiple Entry Type values, and multiple Tag values together
Then each result matches the search term and every selected Entry Type and Tag
And Entries that fail any one of these criteria do not appear.

Scenario 6: Export Book Project in Chapter Order

Given a Book has Chapters in sequential order, Entries associated with those Chapters, and Entries associated directly with the Book
When the user exports the Book project
Then Chapter-associated Entries appear under their respective Chapter headings, with Chapters sorted by ascending internal position when available and otherwise by ascending `created_at`
And Entries associated directly with the Book but not placed under a Chapter appear in a dedicated "General Book Notes" section at the end
And an Entry associated with both a Chapter and the Book appears once, under its Chapter heading.

Scenario 7: Offline Use, Persistence, and Deferred Sync

Given the device has no network connection
When the user creates and organizes an Entry, searches for it, and exports it
Then these operations use local data and complete without network access
And the local mutation is recorded in the durable outbox.

When the user closes and reopens the app while still offline
Then the Entry, its saved Content, and its pending sync operation remain available.

Scenario 8: Create and Structure a General Note

Given the user has an idea to capture
When the user creates a General Note with Content only
Then the Entry is saved as a General Note without requiring a Title, Tag, Book, or Chapter.

When the user later changes its Entry Type, adds a Tag, and associates it with a Book or Chapter
Then those changes are saved and displayed when the Entry is viewed again.

Scenario 9: Export Success and Failure

Given an Entry contains representative accented and non-Latin text
When the user successfully exports the Entry
Then a `.txt` file is available in device-local storage and preserves the text when decoded as UTF-8.

Given a local storage error prevents an export from completing
When the user attempts the export
Then the app reports the failure and does not indicate that the export succeeded.

Scenario 10: Delete an Entry

Given an Entry exists and is linked to another Entry
When the user deletes the Entry
Then the deleted Entry is absent from Entry lists and search results
And no active Entry Link points to the deleted Entry
And the other linked Entry remains available.

Scenario 11: Synchronize Pending Changes

Given the client has pending local Entry and relationship changes and a valid authenticated context
When connectivity returns and synchronization completes
Then the server accepts each valid operation once and returns its acknowledgement
And the client marks only acknowledged operations as synchronized
And the client downloads server changes after its saved cursor and stores the new cursor.

Scenario 12: Retry an Unacknowledged Sync Operation

Given the client sent a sync operation but did not receive its response
When the client retries the same operation identifier
Then the server returns the operation's prior outcome without duplicating its effect
And the client can safely update its outbox from that outcome.

Scenario 13: Detect a Concurrent Update

Given a local Entry was edited using a server revision that is now stale because the Entry changed elsewhere
When the client uploads the edit
Then the server returns a conflict with its current revision and representation
And the client retains the local edit and exposes the conflict without silently replacing either version.

6 Traceability and Outstanding Specification Details

The paired `spec/traceability-matrix.tex` maps every requirement in this SRS to its acceptance criterion and, where applicable, to one or more behavioral scenarios. Functional criteria `AC-F-001` through `AC-F-029`, API criteria `AC-API-001` through `AC-API-014`, and non-functional criteria `AC-NF-001` through `AC-NF-014` each map one-to-one to the corresponding requirement identifier.

The following details are intentionally deferred to design or subsequent project stages:

- authentication mechanism, account creation/recovery, and token/session lifecycle;
- exact JSON field schemas, nullability, server-generated fields, and validation limits;
- API request/response examples, exact status-code selection for each failure, and pagination limits;
- transaction boundaries and dependency ordering for mixed-entity sync batches;
- conflict-resolution UI and the user's resolution choices;
- cursor expiry/recovery behavior and tombstone retention duration;
- deployment, service availability, network performance targets, and supported web browsers;
- whether users can manually reorder Chapters, beyond the export ordering rule specified in REQ-F-028;
- exact text layout, metadata, file naming, and platform destination-selection behavior for exports;
- diagnostic-log redaction and timestamp precision; and
- the precise definition of code included in the REQ-NF-014 coverage scope.

These decisions may refine the API contract but shall not remove the offline client workflow, data-ownership boundary, safe-detach semantics, idempotent retry, or conflict-preservation obligations stated above.